

Índice general

1. Introducción	4
2. Trabajos relacionados y antecedentes	5
3. Metodología	6
3.1. Selección de herramientas: Elección del motor	6
3.1.1. Comparativa de motores gráficos	7
3.2. Arquitectura del Simulador	8
3.2.1. Fundamentos tecnológicos: El <i>Pipeline</i> de Cómputo .	8
3.2.2. El Autómata Celular y el Modelo de Rothermel	9
3.2.3. Comunicación y Sincronización CPU-GPU	13
3.3. Geometría y Datos Topográficos	15
3.3.1. Procesamiento SIG, normalización y adecuación de formatos	16
3.3.2. Renderizado del terreno: Desplazamiento y normales .	16
3.3.3. Extracción a memoria principal y discretización vo- lumétrica	17
3.4. Interacción y Control Dinámico	17
3.4.1. Generación de colisiones topográficas	18
3.4.2. Navegación espacial y cámara libre	18
3.4.3. Sistema de Ignición: Interacción espacial y traducción	18
3.4.4. Interfaz de Usuario y Control Meteorológico	19
3.5. Desafíos Técnicos y Soluciones	20
3.5.1. Sincronización de Espacios de Coordenadas	20
3.5.2. Anomalías Geométricas en la Generación del Relieve .	21
3.5.3. Desincronización de Cámaras en el Contenedor Vo- lumétrico	21
Referencias	22

Índice de figuras

Índice de tablas

3.1. Comparativa de motores gráficos	8
--	---

Capítulo 1

Introducción

Capítulo 2

Trabajos relacionados y antecedentes

Capítulo 3

Metodología

En este capítulo se detalla el proceso de diseño e implementación del simulador volumétrico de incendios forestales. A lo largo del mismo, se justificará la elección del entorno de desarrollo, se desglosará la arquitectura del autómata celular y la implementación de los shaders necesarios, se explicará la integración de datos topográficos reales para la generación del terreno y, finalmente, se comentarán los mecanismos de interacción desarrollados y los principales retos arquitectónicos encontrados.

3.1. Selección de herramientas: Elección del motor

La selección del entorno de desarrollo es una decisión importante que afecta al rendimiento y la escalabilidad del proyecto. A diferencia de otros software, un simulador de incendios forestales en tiempo real basado en autómatas celulares necesita procesar una gran cantidad de datos topográficos y realizar numerosos cálculos para su correcto funcionamiento.

Es por esto que la decisión no se ha basado en las capacidades de renderizado fotorrealista del motor, sino en los siguientes requisitos técnicos:

- **Capacidad de cómputo en paralelo.** Para simular la propagación del fuego a través de la matriz volumétrica es necesario que tenga soporte para la ejecución de *Compute Shaders* permitiendo actualizar el estado de miles de celdas en tiempo real sin saturarse.
- **Control sobre la memoria VRAM.** Es necesario disponer de un gran control de la memoria de video para gestionar los buffers de datos y la sincronización para la comunicación entre la GPU y la CPU sin ocasionar cuellos de botella.
- **Gestión dinámica de datos geoespaciales.** Para el funcionamiento, los datos que se introducirán son Modelos Digitales de Elevaciones

(DEM) y mapas de clasificación de vegetación, por lo que es necesario que se proporcionen interfaces que faciliten su manipulación y conversión a estructuras 3D.

- **Curva de aprendizaje y ecosistema desacoplado.** El entorno debe proporcionar una interfaz clara con una buena documentación, además de una estructura modular que permita separar de forma lógica las distintas interfaces de las que dispondrá el simulador.

3.1.1. Comparativa de motores gráficos

Para tomar una decisión se han comparado los tres motores de desarrollo mas usados ya que son los que tienen mayor soporte y una mayor comunidad.

- **Unreal Engine:** Es el motor lider en rendimiento y ofrece un control de bajo nivel muy profundo mediante C++. Pero para la implementación de *Compute Shaders* necesita una gran cantidad de código de configuración e interactuar con un *pipeline* de renderizado muy complejo. Además de poseer una licencia propietaria.
- **Unity:** Es una de las herramientas más versátiles del mercado, con soporte nativo para la ejecución de *Compute Shaders* en HLSL. Sin embargo, tiene algunas limitaciones para esta simulación, al depender de un recolector de basura (*Garbage Collector*), la transferencia de grandes matrices de datos entre la CPU y la GPU puede sufrir picos de latencia afectando al rendimiento. Además de poseer también una licencia propietaria.
- **Godot:** Es un motor de código abierto con licencia MIT. A partir de su version 4.X ha integrado la API Vulkan la cual resulta muy determinante ya que contiene la interfaz **RenderingDevice**, que permite un acceso directo de bajo nivel a la GPU. Facilitando compilar y ejecutar *Compute Shaders* (escritos en GLSL puro) con un control absoluto sobre la memoria VRAM y los *buffers* de datos. Además ofrece una interfaz clara y sencilla con un sistema de jerarquía de nodos que facilita el aprendizaje y un código desacoplado.

Motor	Soporte <i>Compute Shaders</i>	Acceso VRAM	Sobrecarga (<i>Overhead</i>)	Arquitectura UI	Licencia
Godot 4	Nativo (Vulkan / GLSL)	Alto (RenderingDevice)	Baja	Modular (Nodos)	MIT (Libre)
Unity	Nativo (HLSL)	Medio	Alta (<i>Garbage Collector</i>)	Fragmentada	Propietaria
Unreal	Limitado (Requiere <i>boilerplate</i>)	Alto (Vía C++)	Muy Alta	Pesada (UMG)	Propietaria

Tabla 3.1: Comparativa de los motores gráficos evaluados según los requisitos técnicos del simulador.

3.2. Arquitectura del Simulador

El diseño de la arquitectura de este proyecto se ha dirigido con la necesidad de tratar de manera eficiente una simulación con un gran volumen de datos. Para ello, se ha establecido un modelo de procesamiento híbrido donde la CPU se encarga la lógica de control y la administración del flujo de la aplicación mientras que la GPU actúa como motor físico para ejecutar el cálculo masivo en paralelo. A lo largo de esta sección se explicará cada uno de los componentes de esta arquitectura, desde los fundamentos tecnológicos para su correcta comprensión hasta la implementación técnica en la GPU y sincronización de memorias.

3.2.1. Fundamentos tecnológicos: El *Pipeline* de Cómputo

Comenzamos explicando qué es un *shader*, que es un programa informático que se compila y ejecuta directamente en la GPU. Normalmente su propósito es procesar los efectos de renderizado visual controlando cómo se transforman los vértices de un modelo y calculando la iluminación y el color final de cada píxel en pantalla.

En godot estos *shader* se construyen usando un lenguaje propio denominado *Godot Shading Language*, el cual abstrae gran parte de la configuración de bajo nivel y está optimizado para integrarse de manera fluida con el motor de renderizado visual.

Para el desarrollo de un simulador volumétrico basado en autómatas celulares se necesita actualizar el estado de miles de celdas en cada iteración. Hacer esto de forma secuencial en la CPU generaría un cuello de botella inmenso lo que lo hace inviable para una simulación en tiempo real. Para resolver este problema, la arquitectura del simulador hace uso del concepto GPGPU (*General-Purpose computing on Graphics Processing Units*), el cual permite emplear la enorme cantidad de unidades aritmético-lógicas (ALUs)

de la tarjeta gráfica para resolver operaciones matemáticas de propósito general de forma paralela.

Para hacer uso de esta tecnología se usan los *Compute Shaders*, que a diferencia de los *shaders* tradicionales como los que proporciona Godot, que están estrictamente acoplados al *pipeline* gráfico, estos permiten usar distintos buffers para entradas y salidas, lo cual los hace ideales para hacer los cálculos de la propagación del fuego sobre la matriz de datos

3.2.2. El Autómata Celular y el Modelo de Rothermel

Lograr una simulación forestal equilibrando el coste computacional junto con la calidad visual necesita el desarrollo de varias fases fundamentales, iniciando con la construcción de un contenedor volumétrico que sea capaz de contener la topografía y el fuego en una matriz de simulación renderizada en tiempo real y seguido de la definición del un autómata celular gobernado por las ecuaciones de Rothermel.

Construcción del modelo volumétrico: Raymarching y doble pase

La representación visual del incendio no emplea geometría poligonal tradicional (mallas), sino que utiliza un contenedor volumétrico. Dicho contenedor actúa como un lienzo encargado de renderizar en pantalla la matriz tridimensional que ha calculado el *Compute Shader* en la memoria de vídeo.

Para lograr esta representación sin usar mallas se ha implementado la técnica de *Raymarching* (trazado de rayos volumétrico). Esta técnica consiste en emitir un rayo desde la cámara y que avance a través de un volumen paso a paso, muestreando en cada uno un campo de datos (en este caso, una textura 3D) para acumular color y opacidad.

El desarrollo de esta técnica requiere aislar el volumen de simulación dentro de una (*bounding box*), es decir dentro de la malla de un cubo. Para determinar la trayectoria exacta que debe recorrer cada rayo a través del interior de este cubo, se aplica una estrategia de renderizado de doble pase la cual consiste en hacer dos lecturas, una primera esencial que calcula la posición de las caras traseras del volumen y una segunda que calcula las delanteras para poder obtener así la dirección y tamaño del rayo del *Raymarching*

Antes de comenzar a explicar todo el funcionamiento del contenedor volumétrico y la técnica del doble pase en profundidad es necesario comprender el concepto del *Frame Buffer Object* (FBO). Normalmente la GPU calcula la geometría y los shader y lo vuelca en un buffer que posteriormente se muestra por pantalla, pero al usar un FBO permite que dichos datos pasen

a guardarse en la VRAM sin llegar a mostrarse por pantalla. Esto en Godot se consigue con el nodo **SubViewport** que actúa como un entorno de renderizado aislado que almacena su resultado en una **ViewportTexture** que se puede usar posteriormente para la técnica del doble pase.

El flujo técnico de este contenedor volumétrico opera de la siguiente manera:

1. **Primer pase (FBO y caras traseras):** El entorno de renderizado del **SubViewport** utiliza una cámara interna, sincronizada con la cámara principal del usuario, y un cubo que comparte la misma malla que el cubo en la escena principal. Para renderizar únicamente las caras traseras se usa un shader que se encarga de ocultar las delanteras y ajustar la posición a un color RGB válido (de 0.0 a 1.0). Esto es necesario ya que como se mencionó anteriormente la información del **SubViewport** se guarda en una **ViewportTexture**.
2. **Segundo pase (Caras frontales y cálculo del rayo):** En la escena principal, al igual que en el **SubViewport**, se encuentran la cámara principal y el cubo frontal. Este último posee un *shader* que al contrario del anterior oculta las caras traseras y lee las frontales visibles. Además acepta como parámetro la textura generada por el **SubViewport** para que por cada fragmento al restar la posición de la cara frontal a la trasera se pueda obtener el vector del rayo y saber así la dirección exacta en la que atraviesa el volumen y la distancia total de recorrido.
3. **Avance e interpolación de la matriz (UVW):** Una vez definido el rayo, el algoritmo divide la distancia total en un bucle con un número de pasos definidos por un parámetro y con un tamaño de paso que se calcula teniendo en cuenta este número de pasos y el tamaño total del rayo. Además en cada paso, la posición local actual del rayo, que oscila entre -0.5 y 0.5, se normaliza desplazándola al rango de coordenadas de textura de 0.0 a 1.0.
4. **Muestreo y acumulación de densidad:** Utilizando las coordenadas normalizadas calculadas en cada iteración del bucle, el *shader* consulta el estado de ese voxel en la textura que le ha llegado como parámetro desde el *Compute Shader*, donde un valor de 1.0 indica la existencia de fuego y 0.0 indica espacio vacío. Si el rayo atraviesa un voxel con fuego, se incrementa un contador el cual se usa posteriormente para calcular la densidad de ese fragmento.
5. **Generación del espectro térmico:** Finalmente, con el valor de la densidad acumulada a lo largo del rayo se determina el color del píxel

resultante en pantalla, para ello el sistema aplica una interpolación lineal entre colores anaranjados para las zonas de densidad baja y colores amarillos intensos para las zonas de alta densidad.

Discretización espacial y regla de vecindad

El espacio lógico interior del simulador se divide en una matriz tridimensional que su resolución depende de la lectura directa de Modelos Digitales de Elevaciones (DEM) y sus correspondientes mapas de vegetación. Para cada posición de esta matriz se guarda su estado, cuando se genera el mapa, antes de iniciarse el incendio, puede encontrarse en aire, subsuelo, matorral, árbol, cada uno con diferentes propiedades para la propagación y una vez que se inicia el incendio dicho estado puede actualizarse a Fuego.

La evolución del incendio se decide evaluando los estados de las celdas vecinas y los factores ambientales. Para garantizar una propagación fluida, el algoritmo evalúa una vecindad tridimensional de Moore la cual consiste en iterar un rango matricial de 3x3x3 alrededor de la celda de origen (descartando la posición central), y evaluando el estado de las 26 celdas adyacentes para calcular el riesgo total de ignición.

Integración del modelo físico de propagación

El paso de un voxel intacto a una voxel en llamas sigue una adaptación estocástica del modelo físico de incendios de Rothermel (1972) aplicada a Autómatas Celulares, basándose en la formulación de Alexandridis. En este enfoque, la probabilidad de ignición (P_{ign}) transmitida por una celda vecina en combustión se define mediante la siguiente ecuación matemática:

$$P_{ign} = P_0 \cdot (1 + p_w) \cdot (1 + p_s) \cdot p_h$$

Esta formulación divide la termodinámica de Rothermel en multiplicadores independientes que pueden procesarse en paralelo para millones de voxeles. Cuando una voxel que tenga la posibilidad de arder se encuentra junto a otros que ya están en estado de fuego, el sistema procesa el riesgo de contagio evaluando cada uno de estos factores:

- **Probabilidad base y propensión del combustible (P_0):** Representa el tipo y la carga de combustible. Se asignan diferentes pesos según la densidad de la vegetación presente en la matriz. Por ejemplo el matorral(vegetación baja) es considerado un combustible altamente volátil que arde con rapidez por lo que va a tener una mayor propensión, mientras que el árbol (vegetación alta) tiene una mayor resistencia a la ignición, es decir, una propensión más baja.

- **Factor de viento (p_w):** La influencia del viento se calcula penalizando o bonificando la probabilidad de propagación según si el viento sopla a favor o en contra de la celda y la velocidad del mismo. Se define como:

$$p_w = \exp(V \cdot (\cos(\theta) - 1))$$

Donde V es la velocidad del viento y θ es el ángulo formado entre la dirección del viento y la línea que une el voxel en llamas con su vecino. En la implementación, este factor afecta acelerando la propagación a favor del viento y bajando la probabilidad cuando se encuentre a contraviento.

- **Factor topográfico o de pendiente (p_s):** Representa el ascenso del aire caliente, el cual precalienta el combustible situado cuesta arriba. Su formulación es:

$$p_s = \exp(a \cdot \tan(\theta_s))$$

Donde θ_s es el ángulo de inclinación del terreno. El algoritmo incrementa exponencialmente el riesgo cuando el fuego proviene de un voxel inferior, y lo penaliza en el caso contrario, cuando intenta propagarse de una delta superior a una inferior.

- **Factor de humedad ambiental (p_h):** Funciona como freno general de la propagación ya que el agua contenida en la vegetación debe evaporarse antes de alcanzar el punto de ignición. Se modela como un decaimiento exponencial:

$$p_h = \exp(-c_h \cdot H)$$

Donde H es el índice de humedad relativa (de 0 a 1) y c_h es la constante de sensibilidad del combustible.

Cabe destacar el diseño estructural de la ecuación final. Los factores de viento y pendiente se añaden a la fórmula como multiplicadores de forma $(1 + p_w)$ y $(1 + p_s)$ debido a que actúan como ayudas adicionales, es decir, si las condiciones son favorables aumenta la probabilidad de propagación pero si en cambio van en contra los factores tienden a cero y el multiplicador interno queda en 1, permitiendo que el fuego siga propagándose solo con la probabilidad base (P_0).

Por el contrario, el factor de humedad (p_h) se aplica de forma directa porque actúa como un freno general. Si el entorno está empapado ($p_h \approx 0$), al multiplicar toda la ecuación por cero el fuego se detiene prácticamente por completo, reflejando la dinámica real de los incendios forestales.

Finalmente durante la iteración de la vecindad espacial, el algoritmo suma el riesgo individual calculado de cada una de las celdas vecinas activas,

dando como resultado un valor global de riesgo de ignición para la celda evaluada. Teniendo este valor de probabilidad se calcula un número aleatorio y se determina si se cambia el estado a fuego o no.

3.2.3. Comunicación y Sincronización CPU-GPU

Como se ha mencionado anteriormente, el costoso cálculo de la probabilidad de cada voxel para la propagación pasará a hacerse en un *Compute Shader* para así aprovechar la alta capacidad de procesamiento paralelo de la GPU. Trabajar entre los dos espacios de memoria (la RAM y la VRAM) requiere una la sincronización y transferencia eficiente de datos, que se logra de una manera más sencilla haciendo uso de la clase `RenderingDevice`.

El ciclo de comunicación y ejecución de la simulación se divide en tres fases: la inyección de datos paramétricos, la sincronización de estados mediante *Ping-Pong Buffers* y el despacho masivo de hilos.

Inyección de datos estáticos y parámetros dinámicos

Antes de iniciar el cálculo de la propagación, la CPU debe preparar y transferir el contexto del entorno a la GPU. El flujo de datos se divide en dos categorías según su frecuencia de actualización: estáticos y dinámicos.

Los **datos estáticos** son la topografía y el tipo de vegetación y se evalúan una única vez durante la inicialización del sistema. El *script* generador del terreno recorre la matriz de elevaciones y, en función de la clasificación por colores, construye un `PackedByteArray` que se transfiere a la VRAM, donde el `RenderingDevice` lo ensambla como una textura 3D vinculada al *shader*.

```

1      # Generacion y envio del mapa estatico (mapa_elevaciones.
2      gd)
3      func enviar_mapa_combustibles_a_gpu():
4          var array_combustibles = PackedByteArray()
5          array_combustibles.resize(total_celdas)
6
7          # ... (bucle tridimensional X, Y, Z) ...
8          if estado == ESTADO_MATORRAL:
9              array_combustibles[indice_gpu] = 100
10         elif estado == ESTADO_ARBOL:
11             array_combustibles[indice_gpu] = 255
12
13         # Se inyecta en la GPU a traves del controlador
14         controlador.cargar_mapa_combustibles(
15             array_combustibles)

```

Por otro lado, los **datos dinámicos** (velocidad del viento, dirección y porcentaje de humedad) se actualizan en tiempo real durante la ejecución.

Para lograr esto, se hace uso de *Push Constants*, una técnica que permite enviar pequeños bloques estructurados de memoria directamente a los registros ultrarrápidos de la GPU.

```

1      # Inyeccion de datos dinamicos por frame (controlador.gd)
2      func ejecutar_compute_shader():
3          # Empaquetamos tiempo (semilla) y meteorologia en 32
            bytes exactos
4          var push_constant = PackedFloat32Array([
5              semilla_cpu, 0.0, 0.0, 0.0,
6              velViento, dir_rad, humedad, 0.0
7          ])
8          var bytes = push_constant.to_byte_array()
9
10         # Inyectamos en los registros de la GPU antes de
            ejecutar
11         rd.compute_list_set_push_constant(compute_list, bytes,
            bytes.size())

```

Sincronización de estado: Técnica de *Ping-Pong Buffers*

El modelo de propagación del autómata celular presenta un problema, si un hilo actualiza el estado de una celda de intacta a ardiendo y sobrescribe la matriz instantáneamente, los hilos que aún no hayan finalizado su ciclo leerán el dato ya modificado lo que generaría una condición de carrera que corrompería la validez del modelo.

Para solucionar este conflicto, se implementa la técnica de *Ping-Pong Buffers*, que en lugar de utilizar una única matriz, el sistema reserva dos texturas tridimensionales idénticas en la VRAM rellenándolas inicialmente con ceros que consiste en configurar un *buffer* (el estado anterior) en modo de solo lectura, mientras que el segundo (el estado nuevo) como solo escritura.

- **Buffer A :** Configurado en modo de solo lectura (**readonly**). Contiene la "fotografía" estática del estado actual del bosque que los hilos utilizan para evaluar a sus vecinos.
- **Buffer B:** Configurado en modo de solo escritura (**writeonly**). Es el lienzo en blanco donde cada hilo plasma el resultado de sus cálculos para el siguiente instante de tiempo.

Al finalizar cada orden de despacho computacional, la CPU invierte los punteros lógicos de ambas texturas para el siguiente *frame*.

```

1      # Intercambio de buffers por iteracion (controlador
            .gd)
2      if buffer_a_es_lectura:
3          rd.compute_list_bind_uniform_set(compute_list,
            set_a_lee_b_escribe, 0)
4      else:
5          rd.compute_list_bind_uniform_set(compute_list,
            set_b_lee_a_escribe, 0)

```

```

6      # ... (Despacho a la GPU) ...
7
8      # Preparamos el siguiente turno invirtiendo los
9      roles
10     buffer_a_es_lectura = !buffer_a_es_lectura

```

Configuración de *Workgroups* y Despacho (*Dispatch*)

El último paso de la arquitectura es la ejecución paralela del código (*Compute Shader*). El volumen no se procesa como una lista unidimensional, sino que se subdivide espacialmente para adaptarse a la arquitectura de la tarjeta gráfica y así aprovechar el paralelismo que nos ofrece.

El *Compute Shader* está configurado para agrupar las invocaciones en clústeres cúbicos de $8 \times 8 \times 8$ hilos por grupo. Durante la ejecución, la CPU calcula la cantidad de grupos necesarios dividiendo la resolución total de la matriz entre tamaño del *Workgroup* en cada uno de sus ejes. Finalmente, se envía el comando de despacho (*Dispatch*) a la GPU que instancia simultáneamente los bloques resolviendo la propagación.

```

1      # Division espacial y despacho concurrente (
2      controlador.gd)
3      var grupos_x = ceil(TAMANO_GRID.x / float(
4      TAMANO_WORKGROUP.x))
5      var grupos_y = ceil(TAMANO_GRID.y / float(
6      TAMANO_WORKGROUP.y))
7      var grupos_z = ceil(TAMANO_GRID.z / float(
8      TAMANO_WORKGROUP.z))
9
10     # Orden de ejecucion a la GPU
11     rd.compute_list_dispatch(compute_list, grupos_x,
12     grupos_y, grupos_z)
13     rd.compute_list_end()

```

3.3. Geometría y Datos Topográficos

Un punto clave para un simulador de incendios es tener una buena representación del entorno para así poder conseguir resultados rigurosos. En este proyecto, la geometría y la distribución del combustible se generan de forma procedimental a partir de Modelos Digitales de Elevaciones (DEM) y mapas satelitales de vegetación.

3.3.1. Procesamiento SIG, normalización y adecuación de formatos

Los Modelos Digitales de Elevaciones obtenidos a través de organismos oficiales se encuentran normalmente en formato GeoTIFF (.tif). Aunque este formato es idóneo para Sistemas de Información Geográfica (SIG), su importación directa a motores de renderizado 3D suele presentar numerosos problemas tanto en la lectura como en la precisión debido a limitaciones en la profundidad de bits por canal.

Para solucionar estos problemas y garantizar un relieve topográfico y de buena fidelidad, se necesitó usar el software QGIS, mediante el cual los mapas originales fueron reproyectados espacialmente y delimitados al área de interés del simulador.

Un paso crítico en esta fase fue la normalización matemática de los valores. Las cotas geográficas reales se normalizaron en un rango de 0.0 a 1.0 para que el *shader* encargado de generar la elevación del terreno procese los valores como una variable de color escalar.

Finalmente, el resultado es exportado al formato OpenEXR (.exr). Este formato, un estándar en la industria de los gráficos por ordenador, soporta valores de coma flotante lineales de 32 bits, asegurando que el mapa normalizado no sufra pérdida de precisión decimal antes de inyectarse en el motor gráfico.

3.3.2. Renderizado del terreno: Desplazamiento y normales

Una vez procesados, los mapas se integran en el simulador. La representación visual de la no emplea una malla estática, sino que parte de un plano base que es deformado mediante un *shader* espacial (*spatial*).

Este *shader* se encarga de alterar la coordenada vertical de cada vértice muestreando el mapa de elevaciones introducido. Posteriormente, el valor normalizado leído se multiplica por el diferencial real de altura del terreno, obteniendo así la elevación exacta del relieve original.

```
1      // Lectura y desplazamiento geometrico de la orografia
      (Shader)
2      void vertex() {
3          // Lectura de la altitud normalizada (0.0 a 1.0) en
           el mapa OpenEXR
4          float altura_leida = textureLod(mapa_elevacion, UV,
           0.0).r;
5
6          // Proteccion matematica contra pixeles corruptos (
           NaN/Inf)
```



```

7         float altura_segura = (isinf(altura_leida) || isnan
8             (altura_leida))
9             ? altura_base : altura_leida;
10
11         // Decodificacion: Desplazamiento 3D escalar del
12         vertice
13         VERTEX.y += (altura_segura - altura_base) *
            escala_altura;
    }

```

Un problema generado por alterar los vértices de forma dinámica es que los vectores normales originales del plano dejan de ser válidos, lo que provoca que no se calcule correctamente las sombras y la iluminación. Para solucionarlo, el *shader* recalcula la inclinación real del terreno leyendo la altitud de los cuatro píxeles vecinos y construyendo dos vectores direccionales (el vector tangente y el vector binormal) cuya diferencia angular conforma la pendiente.

Finalmente, el *shader* le asigna el color leyendo el mapa satelital de vegetación y ajustando los parámetros físicos del material (ROUGHNESS y SPECULAR) para simular la reflexión lumínica parecida a la de la realidad.

3.3.3. Extracción a memoria principal y discretización volumétrica

Para construir la matriz lógica del autómata celular, la CPU extrae, descomprime y redimensiona las texturas visuales hacia la memoria RAM. Este volumen espacial se estructura en un (`PackedInt32Array`) para obtener un mejor rendimiento y se procesa siguiendo un proceso:

1. **Cálculo espacial:** Lee el píxel del mapa de elevaciones y lo multiplica por la resolución vertical del contenedor volumétrico para calcular índice exacto de la superficie.
2. **Análisis del color vegetal:** Analiza los canales RGB del píxel. Si el espectro azul es dominante, se almacena como vegetación alta (`ESTADO_ARBOL`), si predomina el rojo, vegetación baja (`ESTADO_MATORRAL`) y el resto de valores se asume como terreno que no puede incendiarse (`ESTADO_SUBSUELO`)

Finalmente, el algoritmo rellena verticalmente los voxels inferiores al que contiene la vegetación como voxels con estado (`ESTADO_SUBSUELO`)

3.4. Interacción y Control Dinámico

Para que el simulador sea accesible y permita al usuario visualizar la escena desde distintos ángulos e interactuar con los factores de simulación se han desarrollado una serie de mecanismos y elementos en Godot.

3.4.1. Generación de colisiones topográficas

El renderizado de la simulación volumétrica y el *shader* que genera el desplazamiento topográfico de las elevaciones no poseen una malla para el subsistema de colisiones de Godot, impidiendo que se pueda interactuar con el terreno mediante el ratón.

Para solucionar esto se ha generado una `HeightMapShape3D` que es un componente que construye dinámicamente un campo de alturas sólido superpuesto a la representación visual. Los valores de altitud originales se ponderan por el factor de diferencia escalar y se ajustan a las cotas límite del mundo. Una vez calculada esta malla de colisión se pueden llevar a cabo los (*RayCasts*) emitidas desde la cámara del usuario para desencadenar la ignición inicial del incendio de forma precisa e interactiva.

3.4.2. Navegación espacial y cámara libre

La simulación cuenta con un sistema de cámara de vuelo libre, que permite al usuario desplazarse sin restricciones espaciales por la escena, utilizando un control clásico mediante teclado y ratón.

La capacidad de observar la propagación de las llamas desde cualquier punto resulta de gran utilidad ya que permite ver en detalle cómo el fuego interactúa con pendientes y tipos de vegetación específicos y obtener así una perspectiva del alcance general de la emergencia.

3.4.3. Sistema de Ignición: Interacción espacial y traducción

Además de la visualización de con el movimiento de la cámara se ha implementado un sistema (*RayCast*) que interactúa con la malla de colisión generada para el terreno y que nos permite iniciar el incendio en cualquier punto de la topografía.

Al registrarse el clic de ignición, el motor proyecta un rayo desde el origen de la cámara en movimiento hacia el mundo físico. Si este rayo intersecta con la malla de colisión topográfica generada anteriormente, el algoritmo recupera el punto de impacto global.

```
1      # Lanzamiento del rayo de interaccion (mapa_elevaciones
2      .gd)
3      var centro_pantalla = get_viewport().get_visible_rect()
4      .size / 2.0
5      var origen = camara.project_ray_origin(centro_pantalla)
6      var direccion = camara.project_ray_normal(
7          centro_pantalla)
8
9      # Proyeccion e interseccion fisica
```

```

7      var impacto = espacio_fisico.intersect_ray(
          parametros_rayo)

```

Debido a que la matriz topográfica opera en un espacio local, la coordenada global del impacto debe ser traducida al sistema de referencia del volumen contenedor. Primero normaliza los vectores para evitar desfases de escala, y seguidamente el algoritmo calcula los índices matemáticos discretos (X, Z) multiplicando la posición local por la resolución base de la matriz.

```

1      # Traducción a matriz y encendido espacial (
          mapa_elevaciones.gd)
2      var pos_local = cubo_frontal.to_local(punto_global)
3      var pos_normalizada = pos_local + Vector3(0.5, 0.5,
          0.5)
4
5      # Calculo del indice matricial (X, Z)
6      var x_matriz = int(pos_normalizada.x * resolucion.x)
7      var z_matriz = int(pos_normalizada.z * resolucion.z)
8
9      # El controlador actualiza la VRAM con el nuevo foco
10     controlador.agregar_fuego_en(x_matriz, y_matriz,
          z_matriz)

```

3.4.4. Interfaz de Usuario y Control Meteorológico

En un entorno real durante un incendio los factores ambientales no son estáticos sino que cambian a lo largo del tiempo que dura el incendio. En el simulador se ha logrado esto mediante una Interfaz de Usuario (UI) superpuesta al visor volumétrico.

Dicha interfaz tiene varios *sliders* representando la velocidad del viento, dirección del viento (en grados) y el índice de humedad. Estos *sliders* están vinculados mediante señales al *script* principal del controlador por lo que cualquier modificación realizada por el usuario es capturada instantáneamente.

Dado que estos parámetros se envían a la tarjeta gráfica empaquetados como *Push Constants* en cada ciclo de iteración, el cambio en las condiciones del entorno tiene un efecto inmediato en el *Compute Shader*, lo que permite observar cómo las llamas se adaptan a las nuevas variables sin necesidad de pausar, recalcular o reiniciar la simulación.

3.5. Desafíos Técnicos y Soluciones

El desarrollo de este simulador ha supuesto diversos retos arquitectónicos y matemáticos. A continuación, se detallan los desafíos más significativos encontrados durante la implementación y las soluciones aplicadas para resolverlos.

El problema: La técnica de *Raymarching* tiene un alto coste computacional. Para una resolución de pantalla estándar (1080p), emitir un rayo por cada píxel y hacerlo dar un gran número de pasos implicaba ejecutar decenas de millones de lecturas a la textura 3D por cada *frame*. Inicialmente, se evaluaba el rayo completo incluso cuando atravesaba zonas densas de fuego o zonas completamente vacías, provocando bajos fotogramas por segundo (FPS).

La solución: Se implementó una estrategia algorítmica de salida temprana en la que en *shader* de la segunda pasada, se estableció un umbral para la densidad acumulada. Al alcanzar este umbral el bucle se interrumpe eliminando los cálculos innecesarios en las regiones internas y posteriores del fuego, y estabilizando el rendimiento en tiempo real.

```
1 // Optimizacion de salida temprana (TwoPassRendering.gdshader)
2 densidad_acumulada += (densidad_punto * multiplicador) *
   tamano_paso;
3
4 if (densidad_acumulada >= 1.0) {
5     break; // El rayo no necesita seguir avanzando
6 }
```

3.5.1. Sincronización de Espacios de Coordenadas

El problema: El sistema de ignición interactiva presentaba una desincronización espacial. Al realizar un clic sobre una montaña, el fuego aparecía en ubicaciones incorrectas o fuera de los límites de la matriz. Esto ocurría porque no se estaban haciendo correctamente los cambios entre los tres espacios matemáticos, las coordenadas globales del motor físico, las coordenadas locales del cubo contenedor y el índice discreto de matriz del autómata celular.

La solución: Se desarrolló un flujo de traducción de coordenadas secuencial. Primero, el punto de impacto global devuelto por el *RayCast* se transforma al espacio local del contenedor. Debido a que las coordenadas de la malla se encuentran entre -0.5 y 0.5, se le suma un vector (0.5, 0.5, 0.5) para obtener coordenadas normalizadas estrictamente positivas entre 0.0 y 1.0. Finalmente, este valor decimal se multiplica por la resolución máxima de la matriz del autómata

-
- 3.5.2. Anomalías Geométricas en la Generación del Relieve
 - 3.5.3. Desincronización de Cámaras en el Contenedor Volumétrico

Bibliografía